

AI-Powered Chapter-wise RAG Learning Platform

Complete Architecture Study, Technology Stack, System Design, and Proof-of-Concept Plan

(Reference Study Based on the Oswaal AI Learning Platform)

Prepared by: Parth

AI Engineer Intern, Shatarupax AI Labs

Document Version 2.0 — Extended Edition with Architecture Diagrams

Table of Contents

1. Purpose of This Document
 2. Executive Summary
 3. Reference Product Overview — Oswaal AI
 4. High-Level System Flow
 5. Core Concepts — Glossary
 6. System Architecture
 7. Document Ingestion Pipeline
 8. Chunking Strategy & Content Hierarchy
 9. Retrieval and Answer Generation Flow
 10. Multi-Agent Design
 11. Learning Modes
 12. LMS Integration
 13. Quiz Generation Approach
 14. Data Model
 15. Recommended Technology Stack
 16. API Design Overview
 17. Security & Privacy Considerations
 18. Testing Strategy
 19. Monitoring & Analytics
 20. Risk Assessment
 21. Phased Development Roadmap
 22. Estimated Effort and Cost
 23. Team & Timeline Plan
 24. Recommended Next Steps
- Appendix A — Sample Prompts
- Appendix B — Glossary Quick Reference

1. Purpose of This Document

This document consolidates the research, architectural analysis, and technical planning carried out for a proposed AI-powered, chapter-wise learning platform for school students (CBSE / ICSE / NCERT curriculum). The study was conducted using Oswaal AI — an existing platform in this space — as a reference point to understand what a serious, production-grade Retrieval-Augmented Generation (RAG) based EdTech product looks like, and to derive a realistic, low-cost path to a working Proof of Concept (POC).

The goal is to move from a general idea (“an AI platform for students”) to a concrete, buildable system with a defined architecture, technology stack, data flow, visual diagrams, and a phased roadmap that can be presented internally for review and approval.

This extended edition expands on the original study with full architecture diagrams, a proposed API design, security and testing considerations, and a more detailed implementation plan, so that the document can serve as a standalone technical reference for the project.

2. Executive Summary

A chapter-wise RAG learning platform allows a student to ask questions and receive explanations, examples, and practice questions that are strictly grounded in their own textbook content — rather than generic answers from the open internet. This is achieved by breaking textbooks into structured chapter/topic-level pieces, converting them into a searchable format, and using a Large Language Model (LLM) to generate answers only from the retrieved, relevant material.

The system is best understood not as a single chatbot, but as an ecosystem of coordinated components:

- A content ingestion pipeline that processes textbook PDFs into structured, searchable knowledge.
- A retrieval layer that finds the most relevant textbook content for a given student question.
- An AI generation layer that produces explanations, quizzes, and summaries grounded in that content.
- A Learning Management System (LMS) layer that tracks student progress, courses, and performance.
- An analytics and recommendation layer that personalizes the learning path over time.

For an initial POC, the recommendation is to scope the system tightly — one board, one class, one subject, 4–5 chapters — and to use a low-cost, largely open-source stack that overlaps significantly with technology already in use on the AI Service Desk project (FastAPI, PostgreSQL, Docker, Groq). This overlap is expected to reduce ramp-up time considerably, since the underlying backend patterns, authentication approach, and deployment method are already familiar from that project.

3. Reference Product Overview — Oswaal AI

Oswaal AI is the AI-driven learning platform of Oswaal Books, an established Indian educational publisher (headquartered in Agra, founded 1984) known for CBSE, ICSE, JEE, and NEET exam preparation material. Oswaal AI extends this catalogue with an interactive, AI-assisted learning layer for NCERT, CBSE, and ICSE books.

Publicly available information indicates that Oswaal AI functions as a chapter-wise RAG platform, allowing students to ask natural-language questions and receive textbook-grounded explanations, worked examples, and practice material, rather than open-ended web search results. This reference point was used purely to understand the category of product being built and to extract reusable architectural patterns — not as a source of proprietary or confidential implementation detail.

4. High-Level System Flow

At a conceptual level, the system follows this sequence for every student interaction:

1. Student selects Board → Class → Subject → Chapter.
2. Student asks a question in natural language.
3. The question is converted into a numerical representation (an embedding).
4. The system searches a vector database, restricted to the selected chapter, for the most relevant textbook chunks.
5. The retrieved chunks are passed to the LLM along with the question.
6. The LLM generates an answer that is grounded strictly in the retrieved content — as an explanation, step-by-step solution, example, or quiz, depending on the mode selected.

Figure 1 — High-Level Student Query Flow

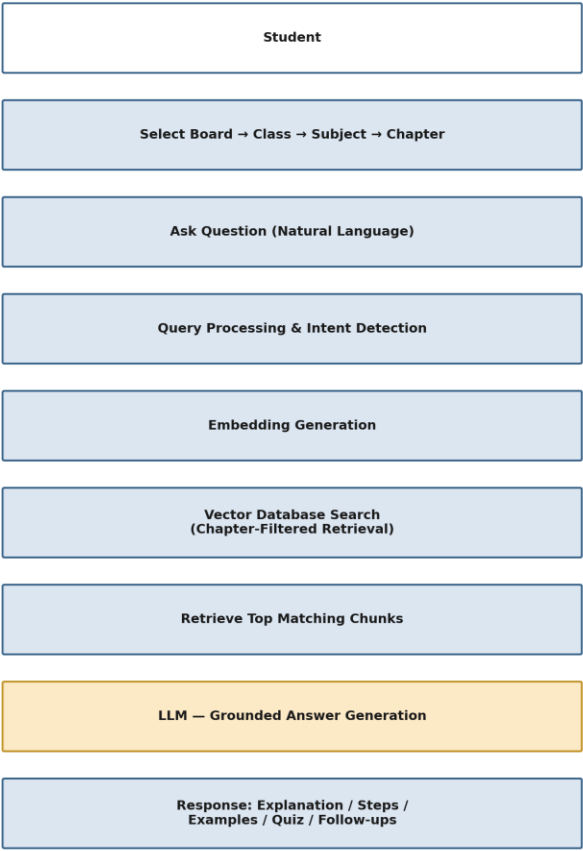


Figure 1 — High-Level Student Query Flow

The key distinction from a general-purpose chatbot is captured below:

Normal Chatbot	Chapter-wise RAG Learning Platform
Answers from general knowledge	Answers grounded in the specific textbook content
Can hallucinate (generate incorrect facts)	Retrieves verified chapter content before answering
No structured learning path	Structured, chapter-wise progression
No memory of student history	Tracks progress, scores, and weak areas
Generic responses for everyone	Personalized explanations and practice
No assessment capability	Generates quizzes and evaluates answers

5. Core Concepts — Glossary

The following terms recur throughout this system and are explained here in plain language, as a reference for anyone reviewing this document without a machine learning background.

5.1 RAG (Retrieval-Augmented Generation)

A technique where relevant information is first retrieved from a trusted source before the AI generates its answer, instead of relying purely on what the model already “knows.” This significantly reduces incorrect or fabricated answers and keeps responses aligned with a specific curriculum.

5.2 OCR (Optical Character Recognition)

A technology that reads text out of scanned images or image-based PDFs and converts it into machine-readable text. It is only required when source PDFs are scanned images; digitally created PDFs can have their text extracted directly without OCR.

5.3 Embeddings

A method of converting text into a list of numbers (a vector) that captures its meaning. Sentences with similar meaning end up with numerically similar vectors, which allows a computer to measure “how similar” two pieces of text are — something it cannot do by comparing raw words alone.

5.4 Vector Database

A specialized database designed to store embeddings and perform fast similarity search — finding the pieces of content whose meaning is closest to a given question. This differs from a conventional database (such as PostgreSQL), which is built for exact-match lookups rather than meaning-based search. Common options include FAISS and ChromaDB (free, local, suitable for a POC) and Pinecone or Azure AI Search (cloud-hosted, suited to production scale).

5.5 Chunking

The process of splitting a large document into smaller, self-contained pieces before it is embedded and stored. This is necessary because language models can only process a limited amount of text at once, and because smaller, well-defined pieces retrieve more accurately than one large, undivided document.

5.6 Metadata & Metadata Filtering

Metadata is descriptive information attached to each chunk — such as board, class, subject, chapter, and topic. Metadata filtering restricts a search to only the relevant chapter or topic, which improves both accuracy and speed by avoiding irrelevant matches from unrelated chapters or books.

5.7 LLM (Large Language Model)

The underlying AI model that understands and generates natural language — for example, Groq-hosted Llama models, OpenAI’s GPT models, or Google’s Gemini models. This is the component that ultimately writes the explanation, quiz, or summary shown to the student.

5.8 LMS (Learning Management System)

The organizational layer that manages users, courses, chapters, assignments, and progress — independent of AI. In this architecture, the LMS decides what a student should be studying and supplies that context to the AI, while the AI decides how to explain or assess it.

5.9 LangChain and LangGraph

LangChain is a development framework that simplifies building a RAG pipeline (connecting retrieval, prompting, and the LLM call). LangGraph extends this to coordinate multiple specialized “agents” that need to work in sequence or collaboration — useful once a system has grown beyond a single retrieval-and-answer flow.

5.10 Multi-Agent System

An architecture where a single large task is divided among several smaller, specialized components (“agents”), each responsible for one function — for example, one agent retrieves content, another generates quizzes, another evaluates answers. This mirrors how a hospital divides work among specialists rather than having one doctor handle every case.

5.11 Tokens

The units in which language models process text — roughly three-quarters of a word each in English. Cost and processing limits for LLMs are measured in tokens, so the size of both the question and the retrieved context directly affects cost and response time.

5.12 JWT (JSON Web Token)

A digitally signed token issued to a user after login, used to verify their identity on subsequent requests without requiring the password to be re-entered each time. This pattern is already in use on the AI Service Desk project for authentication and password reset.

5.13 Redis (Caching)

An in-memory data store used to temporarily hold frequently accessed information (such as active chat sessions) so that the system does not need to repeatedly query the main database, improving response speed.

5.14 Hallucination

A term describing instances where an AI model confidently produces incorrect or fabricated information. RAG is one of the primary techniques used to reduce this risk, by grounding the model’s answers in verified, retrieved source material rather than its own unguided output.

5.15 SQLAlchemy (ORM)

A Python library that maps database tables to Python classes (an Object-Relational Mapper), allowing data to be read and written using Python code rather than hand-written SQL statements.

5.16 Ollama

A tool that allows open-source language models (such as Llama 3) to run locally on a developer's own machine, without depending on a paid cloud API — useful for a zero-cost POC.

6. System Architecture

The full production-grade architecture, as derived from the reference study, is organized into layers as follows:

- Client layer: Web application (Next.js), with a mobile application as a future phase.
- Authentication layer: JWT-based auth for the POC; enterprise identity providers for production scale.
- API layer: FastAPI backend, serving both the LMS and RAG functionality.
- Document management: PDF ingestion, OCR (where needed), chunking, and embedding generation.
- Knowledge storage: A vector database for chunk embeddings, and PostgreSQL for structured data (users, courses, progress).
- AI layer: LLM-based generation, grounded in retrieved chunks, optionally organized into specialized agents.
- Analytics and recommendation layer: Tracking usage and performance to personalize the learning path.

Figure 2 — Full System Architecture

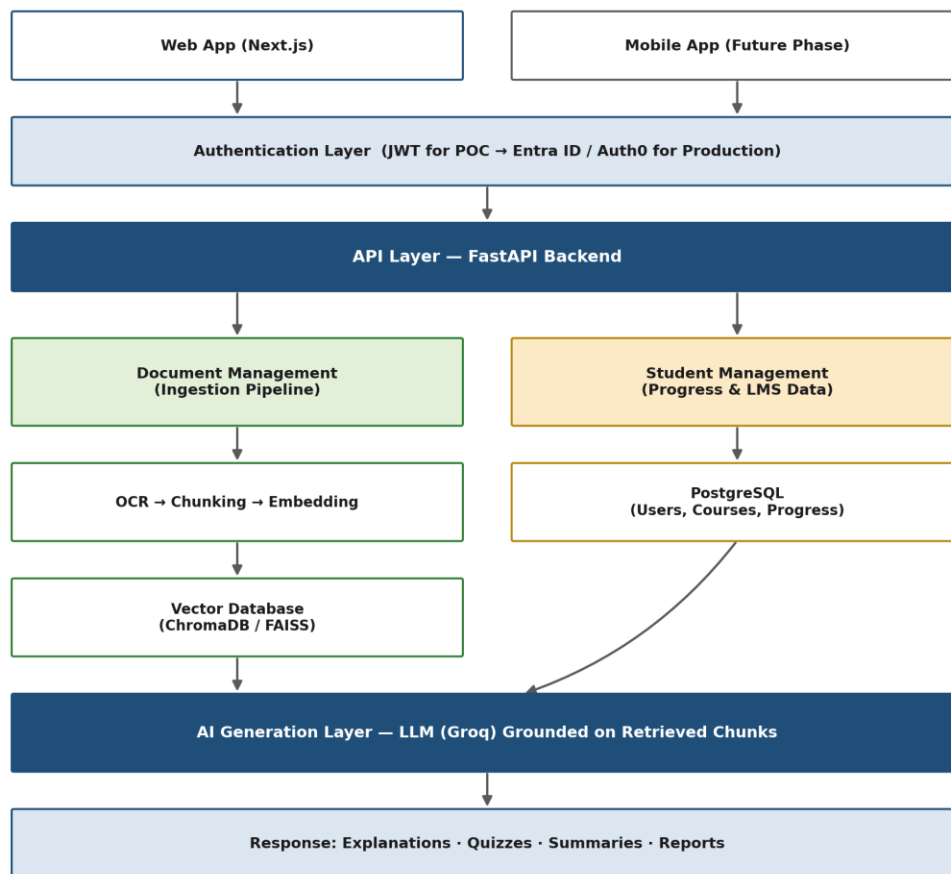


Figure 2 — Full System Architecture

7. Document Ingestion Pipeline

Each textbook proceeds through the following stages before it becomes searchable:

7. Upload of the source PDF.
8. OCR, only if the PDF is a scanned image rather than digitally created.
9. Text extraction and cleaning.
10. Metadata extraction (board, class, subject, chapter, topic, page number).
11. Hierarchical chunking (chapter → topic → paragraph / formula / example / exercise).
12. Embedding generation for each chunk.
13. Storage of the embedding, chunk text, and metadata in the vector database.

Figure 3 — Document Ingestion Pipeline



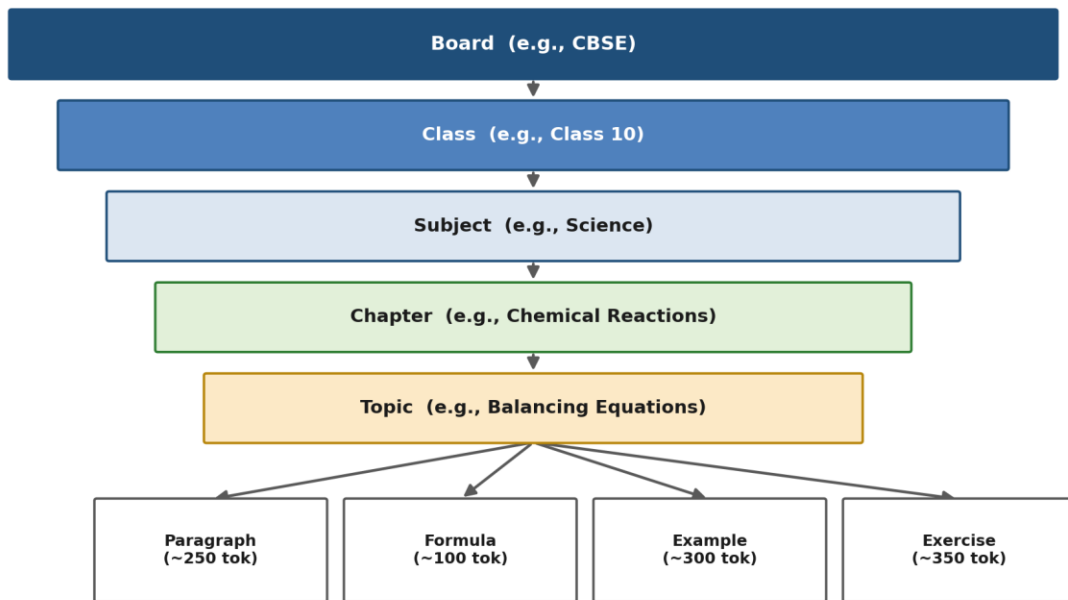
Figure 3 — Document Ingestion Pipeline

Recommended tools for the POC: PyMuPDF or pdfplumber for text extraction from digital PDFs, and Tesseract only where PDFs are scanned images. Extraction quality should be spot-checked manually for the first few chapters, since OCR errors (particularly around scientific notation and formulas) directly degrade retrieval quality downstream.

8. Chunking Strategy & Content Hierarchy

Rather than splitting text into arbitrary fixed-size blocks, content should be chunked hierarchically, preserving the natural structure of the textbook:

- Chapter → Topic → Paragraph, Formula, Example, and Exercise, each treated as its own chunk type.
- Formulas and examples are kept intact within a single chunk and never split mid-content.
- Each chunk carries metadata identifying its board, class, subject, chapter, topic, chunk type, and source page.

Figure 4 — Content Hierarchy Used for Chunking*Figure 4 — Content Hierarchy Used for Chunking*

Recommended chunk sizes, consistent with the reference study:

Content Type	Approx. Token Size
Paragraph	~250 tokens
Formula	~100 tokens
Example	~300 tokens
Exercise	~350 tokens

Sample chunk metadata structure:

```
{  "board": "CBSE",  "class": "10",  "subject": "Science",  "chapter": "Chemical Reactions",  "topic": "Balancing Equations",  "chunk_type": "formula",  "page": 25,  "chunk_id": "C10_SCI_CH1_TOP2_003" }
```

9. Retrieval and Answer Generation Flow

When a student asks a question within a given chapter, the system:

14. Converts the question into an embedding.
15. Searches the vector database, filtered by the student's current board, class, and chapter.
16. Retrieves the top-matching chunks (typically the top 3–5).

17. Passes those chunks, together with the original question, to the LLM within a structured prompt.
18. Returns an answer that is explicitly grounded in the retrieved textbook content, reducing the risk of incorrect or off-syllabus information.

This metadata-filtered search is the single most important accuracy safeguard in the system: without it, a large content library could return a plausible-looking but wrong-chapter chunk, silently degrading answer quality.

10. Multi-Agent Design (Conceptual)

The reference architecture describes the system as a set of specialized agents. For planning purposes, these are grouped as follows:

Student-facing agents

- **Retrieval Agent** — finds the most relevant chapter content for a query.
- **Explanation Agent** — produces simple or detailed explanations, examples, and analogies.
- **Quiz Agent** — generates MCQs, fill-in-the-blank, and assertion-reason questions at varying difficulty levels.
- **Hint Agent** — provides guiding hints rather than direct answers, to encourage independent problem-solving.

Management / backend agents

- **Evaluation Agent** — grades subjective answers and highlights mistakes.
- **Recommendation Agent** — suggests next topics based on performance history.
- **Progress Agent** — tracks completed chapters, accuracy, and study time.
- **Parent / Teacher Agent** — generates periodic performance reports.

Figure 5 — Multi-Agent Coordination (Conceptual)

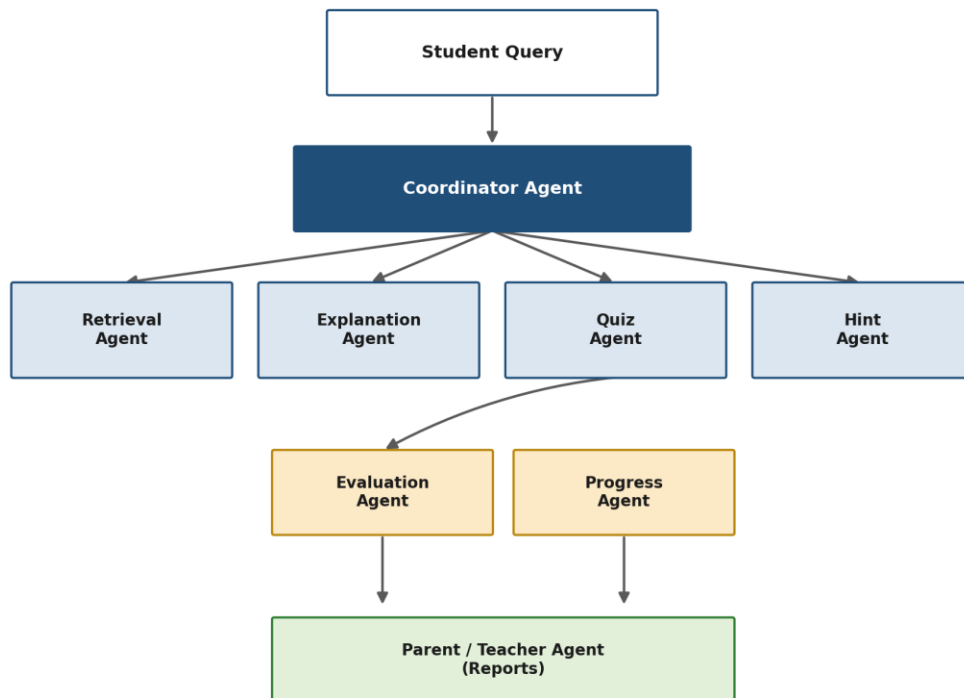


Figure 5 — Multi-Agent Coordination (Conceptual)

For the initial POC, these are best implemented as distinct prompt templates within a single service, rather than as separate orchestrated agents. Full multi-agent orchestration (via LangGraph) is recommended only once the system requires agents to hand off work to one another dynamically.

11. Learning Modes

A single retrieval pipeline can support multiple student-facing modes simply by varying the prompt sent to the LLM:

Mode	Purpose
Learn Mode	Detailed explanation of the selected topic, with examples
Practice Mode	Generation of practice questions
Test Mode	Timed assessment without hints
Revision Mode	Concise summary of key chapter points
Flashcard Mode	Key terms presented as question-answer pairs
Doubt Mode	Open-ended question answering

12. LMS Integration

The Learning Management System determines what a student should study, while the AI layer determines how that content is explained and assessed. In practice, the LMS passes the student's current context (board, class, subject, chapter) to the AI layer so that retrieval and generation remain scoped to the correct material.

Figure 7 — LMS and AI Engine Integration

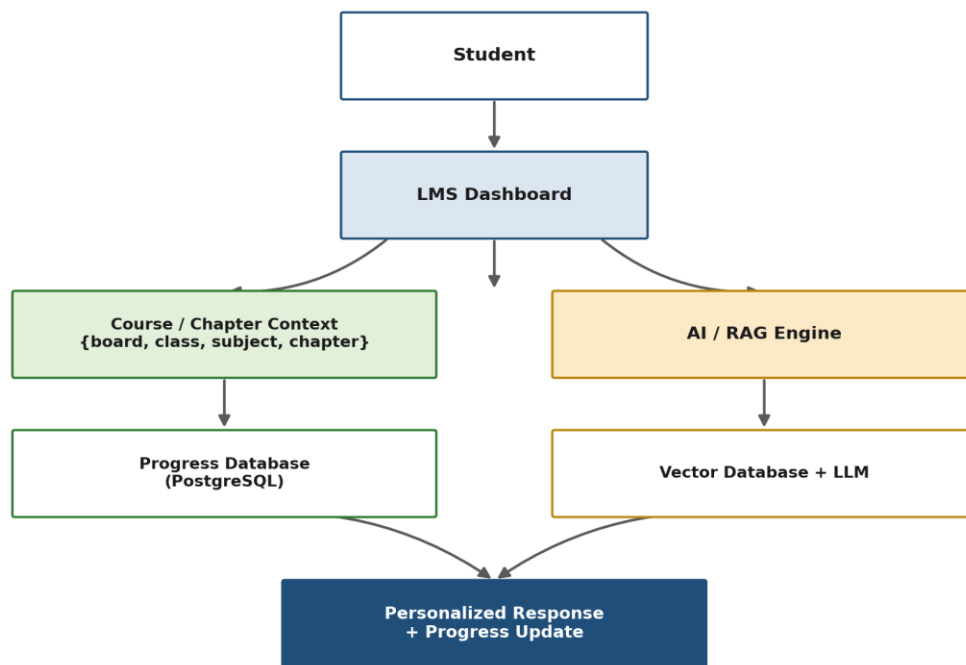


Figure 7 — LMS and AI Engine Integration

12.1 Adaptive Learning Path

A simple, rule-based progression can be used for the POC: a quiz score at or above a defined threshold unlocks the next chapter, while a low score triggers recommended revision, an AI explanation, and a retest. This does not require machine learning to implement, but demonstrates adaptive behavior effectively.

12.2 Contextual Memory

Where feasible, the AI layer should be aware of a student's prior progress (e.g., completed topics, known weak areas) so that explanations can build on what the student has already studied, producing more personalized responses.

12.3 Roles

- Student — learns, practices, and takes assessments.
- Teacher — uploads material, creates assignments, and reviews AI-generated quizzes.
- Parent — views progress reports and weak-area summaries.

- Admin — oversees usage, content, and system-level analytics.

13. Quiz Generation Approach

Quiz generation is implemented as a structured-output task: the LLM is instructed to return a strictly formatted JSON object rather than free-form text, which is then validated before being stored or displayed. This avoids unreliable text parsing and ensures consistent quiz structure (question, options, correct answer, difficulty, and explanation for each item).

Example of the expected output structure:

```
{  "quiz": [    {      "question": "...",      "options": ["A", "B", "C", "D"],      "correct_answer": "A",      "difficulty": "easy",      "explanation": "..."}  ] }
```

If the returned output fails validation (using a Pydantic model, consistent with existing project conventions), the system can retry the request or apply a fallback, ensuring the student-facing experience remains reliable even when the model occasionally returns malformed output.

14. Data Model

The structured data underlying the platform can be organized into the following core entities:

Entity	Key Fields
Users	id, name, email, role, board, class, language
Books	id, title, board, class, subject
Chapters	id, book_id, chapter_name, chapter_order
Topics	id, chapter_id, topic_name
Chunks	id, topic_id, chunk_text, embedding_id, chunk_type, page
Student Progress	id, user_id, chapter_id, status, time_spent, quiz_score, accuracy, last_accessed
Quiz Attempts	id, user_id, chapter_id, quiz_json, student_answers, score, attempted_at
Chat History	id, user_id, chapter_id, question, answer, created_at

Figure 6 — Core Data Model / Entity Relationships

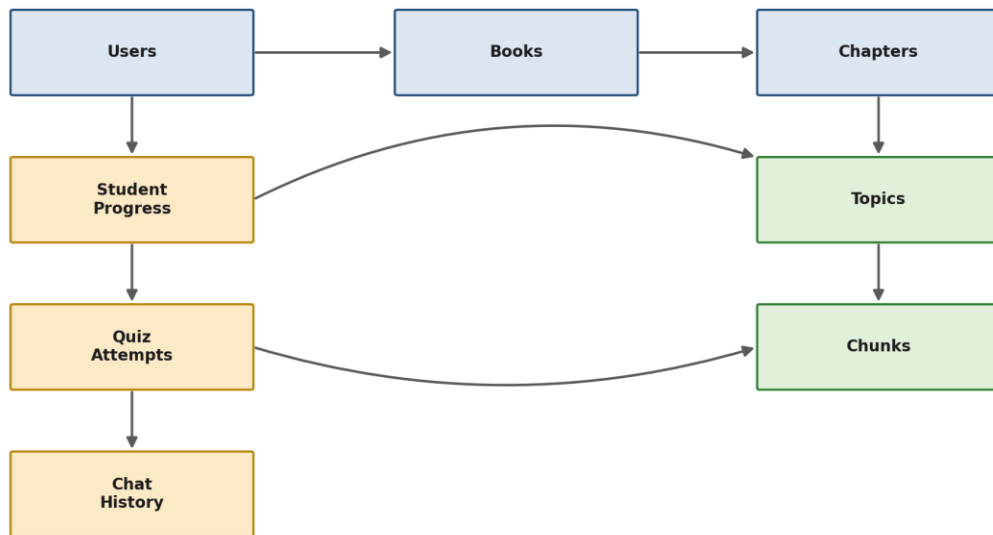


Figure 6 — Core Data Model / Entity Relationships

15. Recommended Technology Stack

The reference material proposes an enterprise-scale, cloud-dependent stack (Azure OpenAI, Azure AI Search, Azure Blob Storage). For an initial, cost-conscious Proof of Concept, a leaner and largely open-source alternative is recommended — one that also overlaps substantially with the technology already used on the AI Service Desk project.

Layer	Reference Stack (Production)	Recommended POC Stack
Frontend	React / Next.js, Flutter (mobile)	Next.js only
Backend	FastAPI	FastAPI
LLM	Azure OpenAI GPT-5	Groq (Llama 3.3 70B)
Embeddings	Azure OpenAI text-embedding-3-large	Open-source sentence-transformers
Vector Database	Pinecone / Azure AI Search	ChromaDB or FAISS
Structured Database	PostgreSQL / Cosmos DB	PostgreSQL
OCR	Azure Document Intelligence	Tesseract (only if scanned PDFs)
File Storage	Azure Blob Storage	Local filesystem
Orchestration	LangGraph / Microsoft Agent Framework	Plain LangChain / direct API calls
Deployment	Azure DevOps, CI/CD	Docker Compose

This scoped stack allows the POC to run entirely without cloud costs, on a single machine, while remaining straightforward to migrate to the production stack at a later stage.

16. API Design Overview

A minimal set of REST endpoints is sufficient to support the POC. These follow the same FastAPI conventions already used on the AI Service Desk project.

Endpoint	Method	Purpose
/auth/login	POST	Authenticate user, issue JWT
/books/upload	POST	Upload a textbook PDF for ingestion
/chapters/{id}/chat	POST	Ask a question within a chapter (RAG chat)
/chapters/{id}/quiz	POST	Generate a quiz for the chapter
/progress/{user_id}	GET	Retrieve a student's progress summary
/quiz/{id}/submit	POST	Submit quiz answers and receive a score

17. Security & Privacy Considerations

- Student accounts should use hashed passwords and JWT-based sessions, consistent with the existing Service Desk authentication pattern.
- Role-based access control should distinguish Student, Teacher, Parent, and Admin permissions at the API layer.
- Uploaded textbook content should be validated (file type, size limits) before ingestion to prevent malicious file uploads.
- Chat history and quiz records contain data about minors in many deployments; access to this data should be restricted to the student, their parent, and authorized teachers only.
- For any future production deployment, data retention and privacy policies should be reviewed against applicable education-data regulations before handling real student data.

18. Testing Strategy

- Unit tests for chunking logic — verifying that formulas and examples are never split mid-content.
- Retrieval accuracy checks — a small labelled set of sample questions per chapter, verifying the correct chunks are retrieved.
- Output validation tests — confirming quiz JSON always conforms to the expected schema, including edge cases where the LLM returns malformed output.
- End-to-end manual testing of the full student flow: login → select chapter → ask question → generate quiz → view progress.

19. Monitoring & Analytics

Even at POC stage, basic tracking of the following is recommended, both to validate the product and to support later investor or management demonstrations:

- Questions asked per chapter, and average response latency.
- Quiz scores and most frequently missed questions.
- Token usage per request (to estimate future production cost).
- Chapters with the lowest average scores, indicating either difficult content or retrieval quality issues.

20. Risk Assessment

Risk	Mitigation
OCR errors on scanned PDFs corrupt formulas/equations	Prefer digitally created PDFs for the POC; manually spot-check extracted text for the chosen chapters
LLM returns malformed quiz JSON	Pydantic validation with automatic retry, and a safe fallback response
Irrelevant chunks retrieved from other chapters	Strict metadata filtering on every retrieval call
Cost escalation if scaled beyond POC without planning	Track token usage from day one; plan the Azure/OpenAI migration path before scaling user count
Scope creep during POC (adding portals, multilingual support, etc.)	Hold strictly to the Phase 1 feature list defined in Section 21

21. Phased Development Roadmap

Phase 1 — Proof of Concept

- Authentication, course and chapter management
- PDF ingestion and chapter-wise RAG chat
- Quiz generation
- Basic progress tracking

Phase 2 — Enhanced Learning

- Teacher portal and assignments
- Adaptive learning paths
- Analytics dashboards
- AI-generated summaries and flashcards

Phase 3 — Production Scale

- Parent portal
- Full multi-agent orchestration
- Gamification, multilingual support, live classes

22. Estimated Effort and Cost (POC Scope)

Since this POC is being built in-house (by an existing team member, using an entirely open-source stack — Groq free/low-cost tier, ChromaDB, sentence-transformers, PostgreSQL, and Docker) rather than outsourced, the effort estimate below reflects a lean, self-build budget covering incidental tooling, minor paid resources, and buffer — not external development fees:

Module	Estimated Cost (INR)
Frontend (Next.js)	₹5,000
Backend API (FastAPI)	₹6,000
PDF ingestion pipeline	₹4,000
Chunking and metadata logic	₹3,000
Vector store integration	₹3,000
RAG chat workflow	₹6,000
Quiz generation	₹4,000
Progress tracking	₹3,000
Testing and bug fixing	₹3,000

Module	Estimated Cost (INR)
Deployment (Docker)	₹2,000
Miscellaneous / buffer (minor API credits, tooling, contingency)	₹1,000

Estimated total: approximately ₹40,000 for the full POC — one board, one class, one subject, 4–5 chapters — built in-house on the open-source stack, with a realistic timeline of three to four weeks of part-time development. This figure assumes no cloud LLM/embedding costs beyond free or low-cost tiers, and no outsourced development effort.

23. Team & Timeline Plan

Week	Focus
Week 1	Environment setup, PDF selection, ingestion pipeline, chunking logic
Week 2	Embedding generation, vector database integration, basic RAG chat
Week 3	Quiz generation, progress tracking, database schema finalization
Week 4	Frontend integration, end-to-end testing, Docker packaging
Week 5 (buffer)	Bug fixes, demo preparation, documentation

24. Recommended Next Steps

19. Finalize the POC scope: one subject, one class, 4–5 chapters (CBSE Class 10 Science is suggested as a demonstrable starting point).
20. Set up the local development stack: FastAPI, PostgreSQL, ChromaDB, and Groq.
21. Build the document ingestion and chunking pipeline for the selected chapters.
22. Implement the core RAG chat flow with metadata-filtered retrieval.
23. Add structured quiz generation and basic progress tracking.
24. Package the POC using Docker Compose for an easy, repeatable demonstration.

Appendix A — Sample Prompts

A.1 Explanation Prompt

You are a helpful tutor. Answer using ONLY the context below. Context: {retrieved_chunks}
 Student question: {question} Provide: a simple explanation, a relevant formula (if applicable), one worked example, and one common mistake students make.

A.2 Quiz Generation Prompt

You are a quiz generator for {board} Class {class} {subject}. Generate exactly 5 MCQs from the context below. Context: {retrieved_chunks} Return ONLY valid JSON, no other text, in this format: { "quiz": [{ "question": "...", "options": ["A","B","C","D"], "correct_answer": "A", "difficulty": "easy|medium|hard", "explanation": "..." }] }

Appendix B — Glossary Quick Reference

Term	One-line Meaning
RAG	Retrieve relevant content, then generate an answer grounded in it
OCR	Convert scanned image text into machine-readable text
Embedding	Numeric representation of text meaning, used for similarity search
Vector Database	Database optimized for similarity search over embeddings
Chunking	Splitting large documents into smaller, retrievable pieces
LLM	The AI model that generates the final natural-language answer
LMS	System managing users, courses, and progress (the “what to learn” layer)
LangChain / LangGraph	Frameworks for building RAG pipelines and multi-agent coordination
Multi-Agent System	Dividing a task among specialized AI components
Token	The unit of text an LLM processes; drives cost and speed
JWT	Signed token used to verify a logged-in user’s identity
Hallucination	AI confidently producing incorrect or fabricated information

End of document.